

Asılılıq inversiyası prinsipi

Asılılıq inversiyası prinsipi (ing. *dependency inversion principle*) iki fikrə əsaslanır:

1. Yuxarı səviyyəli modullar aşağı səviyyəli modullardan asılı olmamalıdır, bunun əvəzinə hər ikisi abstraksiyalardan asılı olmalıdır.
2. Abstraksiyalar detallardan yox, detallar abstraksiyalardan asılı olmalıdır.

Bu fikirləri ətraflı nəzərdən keçirək. Asılılıq inversiyası prinsipinin birinci hissəsinə görə, siniflər digər konkret siniflərdən yox, interfeys və ya abstrakt siniflərdən asılı olaraq yaradılmalıdır. Bir sinif (yuxarı səviyyəli modul) digər sinifdən (aşağı səviyyəli modul) istifadə etsə də, ondan asılı vəziyyətə düşməməlidir. Əks halda konkret sinif daxilindəki dəyişikliklər ondan istifadə edən siniflərin də dəyişməsinə səbəb olacaq. Bu hala yol verməmək üçün, hər iki sinif yalnız abstraksiyalardan asılı olmalıdır. Prinsipin ikinci hissəsinə gəlicə, bu o deməkdir ki, yaradılan interfeyslər və ya abstrakt siniflər onları implementasiya edən konkret siniflərdən asılı olmamalı, tam əksinə olmalıdır. Aşağıdakı nümunəni nəzərdən keçirək.

Elmi-tədqiqat institutlarından biri Selsi ilə verilmiş temperatur göstəricilərini Farengeytə çevirmək istəyir. Onlara xidmət edən proqramlaşdırma şirkətinin əməkdaşı mövcud proqrama aşağıdakı sinifləri daxil edir.

Python

```
class Fahrenheit:

    def convert_temp(self, temp_in_celsius):
        self.temp_in_celsius = temp_in_celsius
        self.split_input()
        if self.is_validated():
            return str(self.temp * 1.8 + 32) + " F"
```



Python

```
↑
    else:
        return "Temperatur Selsi ilə verilməyib!"

    def split_input(self):
        s = self.temp_in_celsius.split()
        self.temp = int(s[0])
        self.symbol = s[1].upper()

    def is_validated(self):
        return self.symbol == "C"

class TempConverter:

    def to_fahrenheit(self, temp_value):
        f = Fahrenheit()
        return f.convert_temp(temp_value)
```

Bir müddət sonra elmi-tədqiqat institutu yeni tələb irəli sürür. Onlar Selsi ilə verilmiş temperatur göstəricilərini Kelvinə də çevirmək istəyirlər. Proqramçı eyni məntiqə hərəkət edərək əlavə Kelvin sinfi yaradır və TempConverter sinfinə yeni to_kelvin metodu əlavə edir.

Python

```
class Fahrenheit:

    def convert_temp(self, temp_in_celsius):
        self.temp_in_celsius = temp_in_celsius
        self.split_input()
        if self.is_validated():
            return str(self.temp * 1.8 + 32) + " F"
↓
```

Python

```

        ↑
    else:
        return "Temperatur Selsi ilə verilməyib!"

    def split_input(self):
        s = self.temp_in_celsius.split()
        self.temp = int(s[0])
        self.symbol = s[1].upper()

    def is_validated(self):
        return self.symbol == "C"

class Kelvin:

    def convert_temp(self, temp_in_celsius):
        self.temp_in_celsius = temp_in_celsius
        self.split_input()
        if self.is_validated():
            return str(self.temp + 273.15) + " K"
        else:
            return "Temperatur Selsi ilə verilməyib!"

    def split_input(self):
        s = self.temp_in_celsius.split()
        self.temp = int(s[0])
        self.symbol = s[1].upper()

    def is_validated(self):
        return self.symbol == "C"

class TempConverter:

    def to_fahrenheit(self, temp_value):
        f = Fahrenheit()
        ↓

```

Python

```

        ↑
        return f.convert_temp(temp_value)

def to_kelvin(self, temp_value):
    k = Kelvin()
    return k.convert_temp(temp_value)

```

Qaynaq koduna diqqət yetirsəniz, burada asılılıq inversiyası prinsipinin pozulduğunu görəcəksiniz. Çünki temperaturun hər yeni ölçü vahidinin (aşağı səviyyəli modul) əlavə olunması ilə `TeamConverter` sinfində (yuxarı səviyyəli modul) dəyişiklik etmək lazım gəlir. Problemi aşağı və yuxarı səviyyəli modulları abstraksiyalardan asılı vəziyyətə gətirməklə həll etmək olar. Proqramçı asılılıq inversiyası prinsipinə əməl etməli olduğunu anladığı vaxtda elmi-tədqiqat institutu Renkin ölçü vahidinə də ehtiyacı olduğunu dilə gətirdi. Aşağıda qaynaq kodunun yenilənmiş variantı verilmişdir.

Python

```

from abc import ABC, abstractmethod

class Temperature(ABC):

    @abstractmethod
    def convert_temp(self, temp_in_celsius):
        pass

    def split_input(self):
        s = self.temp_in_celsius.split()
        self.temp = int(s[0])
        self.symbol = s[1].upper()

    def is_validated(self):
        return self.symbol == "C"

```

Python



```
class Fahrenheit(Temperature):

    def convert_temp(self, temp_in_celsius):
        self.temp_in_celsius = temp_in_celsius
        self.split_input()
        if self.is_validated():
            return str(self.temp * 1.8 + 32) + " F"
        else:
            return "Temperatur Selsi ilə verilməyib!"

class Kelvin(Temperature):

    def convert_temp(self, temp_in_celsius):
        self.temp_in_celsius = temp_in_celsius
        self.split_input()
        if self.is_validated():
            return str(self.temp + 273.15) + " K"
        else:
            return "Temperatur Selsi ilə verilməyib!"

class Rankine(Temperature):

    def convert_temp(self, temp_in_celsius):
        self.temp_in_celsius = temp_in_celsius
        self.split_input()
        if self.is_validated():
            return str(self.temp * 1.8 + 491.67) + " R"
        else:
            return "Temperatur Selsi ilə verilməyib!"
```



Python

```
class TempConverter:
    def __init__(self, unit_of_temp):
        self.unit_of_temp = unit_of_temp

    def convert(self, temp_value):
        return self.unit_of_temp.convert_temp(temp_value)
```

Gördüyünüz kimi, artıq hər dəfə yeni ölçü vahidini əlavə edəndə TempConverter sinfində dəyişiklik etməyə ehtiyac qalmır. TempConverter sinfinin istifadəsini aşağıdakı kliyent kodda nəzərdən keçirin.

Python

```
temp = input("Selsi ilə temperaturu daxil edin: ")
t1 = TempConverter(Fahrenheit())
print(t1.convert(temp))
t2 = TempConverter(Kelvin())
print(t2.convert(temp))
t3 = TempConverter(Rankine())
print(t3.convert(temp))
```

Asılılıq inversiyası prinsipinə əməl etməklə proqramdakı asılılıqları azaltmaq, bununla da qaynaq kodunda ediləcək dəyişiklikləri lokallaşdırmaq mümkündür. Eləcə də interfeyslərin konkret implementasiyaya görə kodlaşdırılmaması sonrakı refaktoring əməliyyatlarının daha asan həyata keçirilməsinə imkan verir.